□ Take-Home Assignment: Multi-Agent Cooking Assistant

Role

Data Scientist / AI Engineer

Estimated Time

6–10 hours

□ Objective

Build a **multi-agent AI system** that can:

- Understand user queries
- Identify intent
- Extract relevant entities
- Route the query to specialized agents
- Return structured, meaningful responses

□ Problem Statement

Users may input natural language queries such as:

- “I want to cook dish like Pav Bhaji”
- “Get me the following ingredients for pasta”
- “How do I prepare biryani?”

Your system should:

1. Identify the **intent**
2. Extract **entities**
3. Route to the correct **agent**
4. Generate a structured response

□ System Architecture

1. Intent Agent

Responsible for:

- Intent classification:
  - `chef_agent`
  - `grocer_agent`
- Entity extraction:
  - Dish name
  - Ingredients
  - Quantities (if applicable)

Expected Pydantic Model:

```
class IntentOutput(BaseModel):
    intent: Literal["chef_agent", "grocer_agent"]
    dish_name: Optional[str]
    ingredients: Optional[List[str]]
```

---

2. Chef Agent

Triggered when intent = `chef_agent`

Responsibilities:

- Generate:
  - Ingredients list
  - Step-by-step recipe
  - Cooking tips
  - Video tutorial links

Expected Output:

```
class ChefOutput(BaseModel):
    dish_name: str
    ingredients: List[str]
    steps: List[str]
    tips: List[str]
    video_links: List[str]
```

---

3. Grocer Agent

Triggered when intent = `grocer_agent`

Responsibilities:

- Fetch ingredient prices (mock APIs allowed)
- Build an optimized cart (minimize cost)

Expected Output:

```
class GroceryItem(BaseModel):
    name: str
    price: float
    store: str

class GrocerOutput(BaseModel):
    items: List[GroceryItem]
    total_cost: float
    store_breakdown: Dict[str, List[GroceryItem]]
```

□ Technical Requirements

Mandatory Stack

- Python
- FastAPI
- Pydantic
- LangChain
- Asyncio
- LLM (OpenAI or equivalent)

□ Functional Requirements

1. API Design

Create:

POST /query

Input:

```
{
  "query": "I want to cook Pav Bhaji"
}
```

Output:

```
{
  "intent": "chef_agent",
  "data": { ... }
}
```

---

2. Async Processing

Use `asyncio` for:

- Parallel API calls (e.g., pricing)
- Efficient agent orchestration

---

3. LLM Usage

Use an LLM for:

- Intent classification
- Entity extraction
- Recipe generation

---

4. Data Validation

- Use **Pydantic models** throughout
- Ensure structured, validated responses

✨Bonus (Optional)

- Caching repeated queries
- Embedding-based intent classification
- Real API integrations
- Unit tests
- Docker setup
- Logging & monitoring

□ Evaluation Criteria

Category	Weight
System Design	20%
Code Quality	20%
Correctness	20%
Async & API Usage	15%
LLM Integration	15%
Bonus Features	10%

□ Deliverables

Submit a **GitHub repository** with:

Required:

- Source code
- README.md
- Setup instructions

README should include:

- Architecture diagram
- Design decisions
- Sample inputs/outputs

Optional:

- Deployment link
- Postman collection

□ Sample Scenarios

Example 1

Input:

```
{
  "query": "I want to cook Pav Bhaji"
}
```

Output:

```
{
  "intent": "chef_agent",
  "data": {
    "dish_name": "Pav Bhaji",
    "ingredients": [...],
    "steps": [...]
  }
}
```

Example 2

Input:

```
{
  "query": "Get me ingredients for pasta"
}
```

Output:

```
{
  "intent": "grocer_agent",
  "data": {
    "items": [...],
    "total_cost": 320
  }
}
```

□ What We're Looking For

- Strong **agent-based system design**
- Clean, modular code
- Effective use of **LLMs**

- Thoughtful engineering trade-offs
- Production-quality structure